

Grouping Unlabelled Data

Grouping Unlabelled Data is a concept in **Machine Learning** where the system organizes data into groups **without having predefined labels**.

This process is mainly used in **Unsupervised Learning**.

In this type of learning, the dataset contains **only input data**, and the algorithm must **discover patterns, similarities, or structures on its own**.

The goal is to **group similar data points together into clusters**.

What is Unlabelled Data?

Unlabelled data means the dataset **does not contain output labels or categories**.

Example dataset:

Customer ID	Age	Purchase Amount
1	22	500
2	45	2000
3	23	450

Here, we only have **data values**, but we don't know the **category of customers**.

Machine learning algorithms analyze this data and **group similar customers together**.

What is Grouping?

Grouping means placing **similar data points into the same cluster**.

This process is called **Clustering (Machine Learning)**.

Each cluster contains data points that are **similar to each other but different from other clusters**.

Example:

Customers can be grouped into:

- Low spending customers
- Medium spending customers
- High spending customers

How Grouping Unlabelled Data Works

The algorithm performs the following steps:

1. Analyze the dataset
2. Measure similarity between data points
3. Place similar data points into the same cluster
4. Separate dissimilar data points into different clusters

The grouping is based on **distance or similarity measures**.

Example

Suppose a company has customer purchase data but does not know customer categories.

The algorithm groups customers like this:

Cluster 1 → Students with small purchases

Cluster 2 → Working professionals with medium purchases

Cluster 3 → High-income customers with large purchases

This helps companies **understand customer behavior**.

Applications of Grouping Unlabelled Data

Grouping unlabelled data is widely used in many real-world applications:

1. **Customer segmentation in marketing**
2. **Document classification**
3. **Image segmentation**
4. **Recommendation systems**
5. **Market basket analysis**

For example, companies like Amazon use grouping techniques to analyze customer buying patterns.

Advantages

- No labeled data is required
 - Helps discover **hidden patterns in data**
 - Useful for **large datasets**
-

Limitations

- Results may vary depending on the algorithm
 - Difficult to evaluate accuracy
 - Choosing the number of groups can be challenging
-

K-Means Clustering

K-Means Clustering is an **unsupervised machine learning algorithm** used to **group unlabeled data into clusters**.

It belongs to Unsupervised Learning in Machine Learning.

The main idea is to divide the dataset into **K different clusters**, where each cluster contains **similar data points**.

Here **K** represents the **number of clusters** chosen by the user.

Basic Idea

The algorithm tries to:

- Group **similar data points together**
- Separate **different data points into different clusters**

Each cluster has a **center point**, called a **centroid**.

The centroid represents the **average position of all points in that cluster**.

Steps of K-Means Algorithm

1. **Choose the number of clusters (K).**

Example:

$K = 3 \rightarrow$ The dataset will be divided into **3 clusters**.

2. **Initialize centroids randomly.**

The algorithm randomly selects **K points as initial cluster centers**.

3. **Assign data points to the nearest centroid.**

Each data point is assigned to the cluster whose centroid is **closest**.

Distance is usually calculated using **Euclidean Distance**.

4. **Update centroids.**

The algorithm calculates the **new centroid** by taking the **average of all points in the cluster**.

5. **Repeat the process.**

Steps 3 and 4 are repeated until the **centroids stop changing**.

At this point, the clusters become **stable**.

Example

Suppose a company has customer data based on **annual spending**.

K-Means can divide customers into groups like:

Cluster 1 → Low spending customers

Cluster 2 → Medium spending customers

Cluster 3 → High spending customers

This helps businesses understand **customer behavior**.

Applications of K-Means Clustering

K-Means is widely used in many real-world applications:

1. **Customer segmentation**
2. **Market analysis**
3. **Image segmentation**
4. **Recommendation systems**
5. **Document clustering**

Companies like Amazon and Netflix use clustering techniques to analyze user behavior.

Advantages

- Simple and easy to implement
 - Works well with **large datasets**
 - Fast clustering algorithm
-

Disadvantages

- Need to choose **K value manually**
 - Sensitive to **initial centroid selection**
 - Does not work well with **irregular cluster shapes**
-

Hierarchical Clustering

Hierarchical Clustering is an **unsupervised machine learning algorithm** used to **group similar data points into clusters**.

It belongs to Unsupervised Learning in Machine Learning.

Unlike **K-Means Clustering**, hierarchical clustering **does not require the number of clusters (K) to be specified in advance**.

Instead, it creates a **tree-like structure of clusters**, which shows how clusters are formed step by step.

This tree structure is called a **Dendrogram**.

Types of Hierarchical Clustering

There are two main types.

1. Agglomerative Clustering (Bottom-Up Approach)

This method starts with **each data point as a separate cluster**.

Then it **merges the closest clusters step by step** until all points belong to one cluster.

Steps

1. Start with each data point as its own cluster
2. Find the **two closest clusters**
3. Merge them into a single cluster
4. Repeat the process until only **one cluster remains**

This is the **most commonly used hierarchical clustering method**.

2. Divisive Clustering (Top-Down Approach)

This method starts with **all data points in one cluster**.

Then the algorithm **splits the cluster into smaller clusters** step by step.

Steps

1. Start with all data points in one cluster
2. Divide the cluster into smaller clusters

3. Continue splitting until each point becomes its own cluster
-

Dendrogram

A **dendrogram** is a **tree diagram** that shows the hierarchical relationship between clusters.

It helps us:

- Visualize how clusters are formed
- Decide the **optimal number of clusters**

Clusters that join **closer to the bottom** are more similar.

Example

Suppose we have data of animals with features like:

- Weight
- Height
- Speed

Hierarchical clustering may group them like:

Cluster 1 → Dog, Wolf

Cluster 2 → Cat, Tiger

Cluster 3 → Horse, Zebra

This grouping is based on **similar characteristics**.

Applications of Hierarchical Clustering

Hierarchical clustering is used in many fields:

1. **Biology (gene classification)**
 2. **Document clustering**
 3. **Image analysis**
 4. **Customer segmentation**
 5. **Social network analysis**
-

Advantages

- No need to choose number of clusters initially
 - Easy to visualize using dendrogram
 - Works well with small datasets
-

Disadvantages

- Computationally expensive for large datasets
 - Once clusters are merged, they **cannot be undone**
 - Sensitive to noise and outliers
-

Probabilistic Clustering

Probabilistic Clustering is an **unsupervised machine learning method** used to group data points based on **probability distributions**.

It belongs to Machine Learning and is commonly used in Unsupervised Learning.

Unlike traditional clustering methods (like K-Means), probabilistic clustering **does not assign a data point strictly to one cluster**.

Instead, it calculates the **probability that a data point belongs to each cluster**.

So a data point can belong to **multiple clusters with different probabilities**.

Basic Idea

In probabilistic clustering, each cluster is represented by a **probability distribution**.

For every data point, the algorithm calculates:

- Probability of belonging to Cluster 1
- Probability of belonging to Cluster 2
- Probability of belonging to Cluster 3

The cluster with the **highest probability** is usually selected as the final assignment.

Example

Suppose we want to classify animals based on features like:

- Weight
- Height
- Speed

For one animal the probabilities may be:

Cluster Probability

Cluster 1 0.6

Cluster 2 0.3

Cluster 3 0.1

This means the animal **most likely belongs to Cluster 1**, but it still has a small relationship with the other clusters.

Common Model Used

A common probabilistic clustering model is the **Gaussian Mixture Model (GMM)**.

In this model:

- Each cluster follows a **Gaussian (normal) distribution**.
- Data points are assigned probabilities for each cluster.

This method works better when clusters **overlap or have complex shapes**.

Steps of Probabilistic Clustering

1. Assume the number of clusters.
 2. Assign **initial probabilities** for each data point.
 3. Estimate the **parameters of probability distributions**.
 4. Update cluster probabilities iteratively.
 5. Repeat until the model converges.
-

Applications of Probabilistic Clustering

Probabilistic clustering is used in many areas:

1. **Image segmentation**
 2. **Speech recognition**
 3. **Customer behavior analysis**
 4. **Document classification**
 5. **Medical data analysis**
-

Advantages

- Handles **overlapping clusters** well
- Provides **probability-based results**
- More flexible than simple clustering methods

Disadvantages

- Computationally more complex
 - Requires strong statistical assumptions
 - Slower for very large datasets
-

Probabilistic Clustering

Probabilistic Clustering is a clustering technique used in Machine Learning under Unsupervised Learning.

It groups data points based on **probability distributions instead of fixed boundaries**.

In many clustering algorithms like K-Means, a data point belongs to **only one cluster**. But in probabilistic clustering, a data point can belong to **multiple clusters with different probabilities**.

This means the algorithm calculates **how likely it is that a data point belongs to each cluster**.

The cluster with the **highest probability** is usually selected as the final cluster.

Basic Concept

In probabilistic clustering:

- Each cluster is represented by a **probability model**.
- Each data point has a **probability of belonging to every cluster**.
- The algorithm uses statistical methods to **estimate these probabilities**.

This approach works well when **clusters overlap** or are not clearly separated.

Example

Suppose we want to group customers based on:

- Age
- Income
- Spending behavior

For a particular customer, the probabilities might be:

Cluster	Probability
Cluster 1	0.55
Cluster 2	0.30
Cluster 3	0.15

This means the customer is **most likely in Cluster 1**, but still has some relation with other clusters.

Gaussian Mixture Model

A common probabilistic clustering technique is the Gaussian Mixture Model.

In this model:

- Each cluster follows a **Gaussian (normal) distribution**.
- The dataset is represented as a **mixture of multiple Gaussian distributions**.

The algorithm calculates the **probability that each data point belongs to each Gaussian cluster**.

Steps of Probabilistic Clustering

1. Assume the number of clusters.
 2. Initialize the parameters of probability distributions.
 3. Calculate the probability that each data point belongs to each cluster.
 4. Update the parameters of clusters based on probabilities.
 5. Repeat the process until the results stabilize.
-

Applications of Probabilistic Clustering

Probabilistic clustering is used in many real-world applications:

- **Customer segmentation in marketing**
 - **Image segmentation**
 - **Speech recognition systems**
 - **Document classification**
 - **Medical data analysis**
-

Advantages

- Handles **overlapping clusters** effectively

- Provides **probability-based cluster membership**
 - More flexible than traditional clustering methods
-

Disadvantages

- Computationally more complex
 - Requires statistical assumptions
 - Slower for very large datasets
-

Association Rule Mining

Association Rule Mining is a technique in Machine Learning used to **discover relationships or patterns between items in large datasets**.

It is commonly used in **market basket analysis**, where businesses analyze **customer purchase behavior**.

The goal is to find **rules that show how items are associated with each other**.

Example rule:

If a customer buys bread, they are likely to buy butter.

This relationship is written as:

Bread → Butter

This means when **bread is purchased**, there is a high chance **butter will also be purchased**.

Key Idea

Association Rule Mining tries to answer questions like:

- What products are often **bought together**?
- What items are **strongly related**?
- What patterns exist in transaction data?

Companies like Amazon use this technique to recommend products.

For example:

"Customers who bought this item also bought..."

Important Terms in Association Rule Mining

1. Support

Support measures **how frequently an item appears in the dataset**.

It shows how often a rule occurs in all transactions.

Example:

If **100 transactions** exist and **20 contain bread**, the support of bread is **20%**.

2. Confidence

Confidence measures **how likely item B is purchased when item A is purchased.**

Example:

Bread → Butter

If 20 people buy bread and 15 also buy butter, confidence = **75%**.

This means **75% of bread buyers also buy butter.**

3. Lift

Lift measures **how strong the association rule is.**

- Lift > 1 → strong relationship
 - Lift = 1 → no relationship
 - Lift < 1 → weak relationship
-

Example

Consider a supermarket dataset:

Transaction Items Bought

T1	Bread, Milk
T2	Bread, Butter
T3	Milk, Butter
T4	Bread, Milk, Butter

From this data we can generate rules like:

- Bread → Butter
- Milk → Bread
- Bread → Milk

These rules help businesses **improve product placement and marketing strategies.**

Applications of Association Rule Mining

Association rule mining is widely used in:

1. **Market basket analysis**
 2. **Product recommendation systems**
 3. **Customer behavior analysis**
 4. **Web usage mining**
 5. **Fraud detection**
-

Algorithms Used

Some algorithms used for association rule mining include:

- **Apriori Algorithm**
- **FP-Growth Algorithm**

These algorithms help **find frequent itemsets efficiently**.

Advantages

- Helps discover **hidden patterns in data**
 - Useful for **business decision making**
 - Improves **recommendation systems**
-

Disadvantages

- Requires large datasets
 - Can generate **too many rules**
 - Computationally expensive
-

Apriori Algorithm

The **Apriori Algorithm** is a popular algorithm used in **Data Mining** and **Machine Learning** to find **frequent itemsets and association rules** in large datasets.

It is mainly used in **market basket analysis**, where businesses analyze which products are **frequently bought together**.

The Apriori algorithm works on the principle that:

If an itemset is frequent, then all of its subsets must also be frequent.

This principle is called the **Apriori property**.

Example:

If the itemset **{Bread, Butter, Milk}** is frequent, then:

- {Bread, Butter}
- {Bread, Milk}
- {Butter, Milk}

must also be frequent.

Steps of Apriori Algorithm

1. Set minimum support value.

Only itemsets with support greater than this value will be considered frequent.

2. Find frequent 1-itemsets.

Count how many times each item appears in transactions.

Example:

Bread = 4

Milk = 3

Butter = 2

3. Generate candidate itemsets.

Combine frequent itemsets to create larger itemsets.

Example:

{Bread, Milk}
{Bread, Butter}

4. Calculate support for candidate itemsets.

Remove itemsets that do not meet the **minimum support threshold**.

5. Repeat the process

Continue generating larger itemsets until **no more frequent itemsets are found**.

Example

Transactions:

TID Items

T1 Bread, Milk

T2 Bread, Butter

T3 Milk, Butter

T4 Bread, Milk, Butter

Apriori finds frequent itemsets like:

- {Bread}
- {Milk}
- {Bread, Milk}

Then it generates rules such as:

Bread → Milk

Advantages

- Easy to understand
 - Works well for **small datasets**
-

Disadvantages

- Generates **many candidate itemsets**
 - Slow for **large datasets**
-

FP-Growth Algorithm

The **FP-Growth Algorithm (Frequent Pattern Growth)** is an improved method used to find **frequent itemsets efficiently**.

It was developed to overcome the **limitations of the Apriori algorithm**.

Unlike Apriori, FP-Growth **does not generate candidate itemsets**, which makes it **much faster**.

Main Idea

FP-Growth uses a special data structure called **FP-Tree (Frequent Pattern Tree)** to store transaction data in a compressed form.

The algorithm then extracts frequent patterns directly from this tree.

Steps of FP-Growth Algorithm

1. Scan the dataset and find **frequent items**.
 2. Build an **FP-Tree** structure.
 3. Store transactions in the tree based on frequency order.
 4. Extract **frequent itemsets** from the tree.
-

Example

Suppose customers buy:

Transaction Items

T1	Bread, Milk
T2	Bread, Butter
T3	Milk, Butter
T4	Bread, Milk, Butter

FP-Growth compresses this data into an **FP-Tree** and finds frequent patterns faster.

Difference Between Apriori and FP-Growth

Feature	Apriori	FP-Growth
Candidate Generation	Yes	No
Speed	Slow	Faster
Data Structure	Multiple scans	FP-Tree
Efficiency	Lower	Higher

Short Exam Definition

Apriori Algorithm:

An algorithm used to find frequent itemsets and generate association rules using the Apriori property.

FP-Growth Algorithm:

A fast algorithm that finds frequent patterns using an FP-Tree structure without generating candidate itemsets.

Gaussian Mixture Model (GMM)

The **Gaussian Mixture Model (GMM)** is a **probabilistic clustering method** used in Machine Learning under Unsupervised Learning.

It assumes that the dataset is generated from a **mixture of several Gaussian (normal) distributions**.

Each Gaussian distribution represents a **different cluster** in the data.

Instead of assigning a data point to **only one cluster**, GMM calculates the **probability that a data point belongs to each cluster**.

This makes GMM more flexible than algorithms like **K-Means Clustering**.

Gaussian Distribution

A Gaussian distribution is also called a **normal distribution**.

It is a bell-shaped curve that describes how values are distributed around a mean.

The probability density function of a Gaussian distribution is:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

sigma
mu

-10-8-6-4-2246810-0.5-0.4-0.3-0.2-0.1

Where:

- **μ (mu)** = Mean (center of the distribution)
 - **σ² (sigma square)** = Variance (spread of the data)
 - **x** = Data point
-

Basic Idea of GMM

In GMM:

- Each cluster follows a **Gaussian distribution**.
- The dataset is modeled as a **combination of multiple Gaussian distributions**.
- Every data point has a **probability of belonging to each cluster**.

Example:

Data Point Cluster 1 Cluster 2

A	0.7	0.3
B	0.4	0.6

This means point **A is more likely in Cluster 1**, while **B is more likely in Cluster 2**.

Working of GMM

GMM usually works using the **Expectation–Maximization Algorithm**.

Steps

1. Choose the number of clusters.
 2. Initialize parameters (mean, variance, probability).
 3. **Expectation Step**: Calculate probability of each data point belonging to clusters.
 4. **Maximization Step**: Update cluster parameters based on probabilities.
 5. Repeat until the model converges.
-

Applications of GMM

Gaussian Mixture Models are used in many areas:

- **Image segmentation**
 - **Speech recognition**
 - **Customer segmentation**
 - **Pattern recognition**
 - **Anomaly detection**
-

Advantages

- Handles **overlapping clusters** well
 - Provides **probabilistic cluster membership**
 - More flexible than K-Means
-

Disadvantages

- Computationally more complex
 - Requires selecting the number of clusters
 - Sensitive to initialization
-

Short Exam Definition

Gaussian Mixture Model (GMM) is a probabilistic clustering algorithm that assumes data is generated from a mixture of Gaussian distributions and assigns probabilities of belonging to each cluster.

Statistical Learning Theory

Statistical Learning Theory (SLT) is a theoretical framework in Machine Learning that explains **how learning algorithms make predictions from data** and how well they **generalize to new unseen data**.

It focuses on finding a model that **minimizes prediction error** and performs well not only on training data but also on **future data**.

The main goal of Statistical Learning Theory is to **build models that balance accuracy and generalization**.

Important concepts in Statistical Learning Theory include:

- Risk Minimization Principle
- Bias–Variance Trade-off
- Overfitting

(a) Risk Minimization Principle

The **Risk Minimization Principle** states that a learning algorithm should choose a model that **minimizes prediction error (risk)**.

In machine learning, **risk** refers to the **expected error of a model** when predicting new data.

There are two main types of risk:

1. Empirical Risk

Empirical risk is the **error calculated using the training dataset**.

The goal is to minimize this training error.

2. Structural Risk

Structural risk considers both:

- Training error
- Model complexity

It tries to **avoid overly complex models** that may cause overfitting.

A good machine learning model minimizes **both empirical risk and structural risk**.

(b) Bias–Variance Trade-off

The **Bias–Variance Trade-off** is an important concept that explains the balance between **model simplicity and model flexibility**.

Bias

Bias is the error caused by **simplifying assumptions in the model**.

High bias means the model is **too simple** and cannot capture the patterns in the data.

Example:

Using a straight line to model complex data.

This leads to **underfitting**.

Variance

Variance is the error caused by **too much sensitivity to training data**.

High variance means the model **fits the training data very closely** but performs poorly on new data.

This leads to **overfitting**.

Trade-off

A good machine learning model must maintain a **balance between bias and variance**:

- High Bias → Underfitting
- High Variance → Overfitting

The best model has **moderate bias and moderate variance**.

(c) Overfitting

Overfitting occurs when a machine learning model **learns the training data too well**, including noise and random patterns.

As a result:

- The model performs **very well on training data**
- But performs **poorly on new or unseen data**

This happens when the model becomes **too complex**.

Example:

A very deep decision tree that memorizes training data.

Ways to Reduce Overfitting

Overfitting can be reduced using techniques such as:

- Using **more training data**
 - Applying **regularization techniques**
 - Using **simpler models**
 - Applying **cross-validation**
-

1. Describe feature extraction techniques in detail: (a) Principal Component Analysis (PCA) (b) Singular Value Decomposition (SVD)

Feature Extraction

Feature Extraction is a technique in Machine Learning used to **reduce the number of input variables while keeping the most important information** in the dataset.

In many real-world datasets, there are **many features (variables)**. Some of them may be:

- redundant
- irrelevant
- noisy

Feature extraction transforms the original data into a **new set of features** that capture the **most useful information**.

Benefits of feature extraction:

- Reduces **dimensionality of data**
- Improves **model performance**
- Reduces **computational cost**
- Removes **noise and redundancy**

Two important feature extraction techniques are:

- Principal Component Analysis (PCA)
 - Singular Value Decomposition (SVD)
-

(a) Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a statistical technique used to **reduce the dimensionality of a dataset while preserving most of the variance in the data**.

PCA converts the original features into **new variables called principal components**.

These components are:

- uncorrelated with each other
- ordered by importance

The **first principal component** captures the **maximum variance** in the data.
The second component captures the **next highest variance**, and so on.

Working of PCA

Steps of PCA:

1. **Standardize the dataset**
Normalize the data so that all features have the same scale.
 2. **Compute the covariance matrix**
This shows how variables are related to each other.
 3. **Calculate eigenvalues and eigenvectors**
 4. **Select principal components**
Choose the components with the largest eigenvalues.
 5. **Transform the dataset**
Convert the original data into the new reduced feature space.
-

Example

Suppose a dataset contains:

- height
- weight
- age
- income

PCA may reduce these **4 variables into 2 principal components** that still contain most of the useful information.

Applications of PCA

PCA is used in:

- **Image compression**
- **Face recognition**
- **Data visualization**
- **Noise reduction**

(b) Singular Value Decomposition (SVD)

Singular Value Decomposition (SVD) is a mathematical technique used to **decompose a matrix into three smaller matrices**.

It is widely used in **data science, machine learning, and signal processing**.

SVD helps in **reducing dimensionality and extracting important features from data**.

The matrix decomposition is represented as:

$$A = U \Sigma V^T$$

Where:

- **A** = Original data matrix
- **U** = Left singular vectors
- **Σ (Sigma)** = Diagonal matrix containing singular values
- **V^T** = Right singular vectors

These matrices help represent the **important structure of the data**.

Working of SVD

Steps involved in SVD:

1. Represent the dataset as a **matrix**.
 2. Decompose the matrix into **U, Σ , and V^T** .
 3. Keep only the **largest singular values**.
 4. Reconstruct the dataset with **reduced dimensions**.
-

Applications of SVD

SVD is widely used in:

- **Recommendation systems**
- **Natural Language Processing**
- **Image compression**
- **Topic modeling**

- **Latent Semantic Analysis**

For example, recommendation systems like Netflix use SVD-based methods to recommend movies.

Difference Between PCA and SVD

Feature	PCA	SVD
Type	Statistical method	Matrix factorization method
Purpose	Dimensionality reduction	Matrix decomposition
Data	Uses covariance matrix	Works directly on data matrix
Application	Feature extraction	Data compression and recommendation

Short Conclusion (for exams)

Feature extraction techniques such as Principal Component Analysis (PCA) and Singular Value Decomposition (SVD) are used to reduce the dimensionality of datasets while preserving important information. PCA transforms features into principal components based on variance, while SVD decomposes a matrix into three matrices to reveal the underlying structure of the data.